INTELLIGENT SYSTEMS

Problem Statements
Participant Edition

Research • Engineer • Build • Impact

16 Advanced Problem Statements

5 Application Domains • Software-Only • Open Architecture

Participant Release
This booklet contains the official problem statements for registered hackathon participants. Each problem describes the real-world challenge and its known constraints. The solution path is intentionally open: no programming language, framework, database, cloud provider, algorithm, architecture, or technology stack is prescribed.

Important: All problem statements are software-only. No hardware, embedded systems, or IoT dependency is required.

Domains: Healthcare & Well-being | Smart Cities & Urban Infrastructure | Sustainability & Environment | Cybersecurity & Digital Trust | Enterprise Productivity & Intelligent Automation

About the Challenge

The **Intelligent Systems** track focuses on difficult software problems where the central challenge lies in systems reasoning, computational methods, reliability, security, or intelligent decision-making rather than simply assembling screens and APIs.

The sixteen problems span five application domains. Teams are free to choose the implementation approach that best fits the problem. The problem statements intentionally leave the architecture, algorithms, frameworks, infrastructure, and technology choices open.

What Is Open?

There is no mandated programming language, database, cloud provider, framework, algorithm, model, architecture, protocol, or infrastructure product. Selecting appropriate technologies and justifying those choices is part of the engineering challenge.

How to Read a Problem

For each problem, focus on two things:

1. the real-world problem and objective that must be addressed; and
2. the known constraints that define the boundaries of a valid solution.

The implementation and solution strategy are deliberately left to the team.

Problem Statement Index

Code	Domain	Problem Statement
H-01	**Healthcare**	Federated Clinical Query Fabric
H-02	**Healthcare**	Real-Time Clinical Resource Transaction System
H-03	**Healthcare**	Privacy-Preserving Collaborative Medical Imaging Network
S-01	**Smart Cities**	City-Scale Real-Time Event Fabric
S-02	**Smart Cities**	Autonomous Cloud-Edge Computing Fabric
S-03	**Smart Cities**	Urban Infrastructure Cascade Simulator
E-01	**Sustainability**	Decentralized Environmental Data Provenance Network
E-02	**Sustainability**	Semantic Environmental Data Compression Network
E-03	**Sustainability**	Climate Adaptation Strategy Simulator
C-01	**Cybersecurity**	Real-Time Software Supply Chain Graph
C-02	**Cybersecurity**	Adaptive Cyber Deception Infrastructure
C-03	**Cybersecurity**	Cryptographic Agility & Post-Quantum Migration Graph
C-04	**Cybersecurity**	Spatial Cyber Threat Reconstruction Engine
P-01	**Enterprise Productivity**	Enterprise Process Genome
P-02	**Enterprise Productivity**	Distributed Transaction Coordinator for Human Workflows
P-03	**Enterprise Productivity**	Natural Language to Verified Workflow Compiler

Choose the problem whose constraints demand the strongest technical reasoning from your team.

Contents

Healthcare & Well-being

Healthcare & Well-being H-01

Federated Clinical Query Fabric

Problem Statement

Healthcare data is fragmented across hospitals, diagnostic laboratories, pharmacies, insurance providers, and research institutions. Even when multiple organizations possess complementary information about the same clinical problem, privacy policies, institutional boundaries, and data ownership prevent them from creating a centralized database.

Develop a **Federated Clinical Query Fabric** that allows authorized institutions to execute complex analytical queries across distributed healthcare databases without transferring the underlying patient records to a central repository.

For example, an authorized researcher should be able to ask for the number of patients across participating institutions who satisfy a defined combination of clinical conditions and treatment history, while each institution retains control of its underlying data.

Known Constraints

- Support heterogeneous institutional schemas and data representations.
- Execute computation as close to the source data as possible.
- Provide fine-grained authorization for queries and data operations.
- Preserve data provenance throughout query execution.
- Handle node failures and network partitions without silently producing invalid results.
- Provide a mechanism for validating or auditing returned results.
- Demonstrate performance against a centralized baseline.

Healthcare & Well-being H-02

Real-Time Clinical Resource Transaction System

Problem Statement

Hospitals operate thousands of concurrent resource transactions involving beds, doctors, operating rooms, emergency departments, medications, diagnostic equipment, and patient transfers. A decision made for one patient can invalidate an allocation made by another department.
Develop a **Real-Time Clinical Resource Transaction System** that treats hospital operations as a continuously changing distributed transactional environment. The system must maintain a coherent operational state while independent services and human operators issue concurrent allocation, reservation, transfer, cancellation, and escalation requests.

Known Constraints

- Model resource allocation as explicit transactions or durable workflows.
- Handle concurrent conflicting requests deterministically.
- Support duplicate and out-of-order events.
- Recover from partial service failures.
- Provide auditability for every resource decision.
- Support compensation or rollback for partially completed workflows.
- Demonstrate behavior under simulated high-concurrency workloads.

Healthcare & Well-being H-03

Privacy-Preserving Collaborative Medical Imaging Network

Problem Statement

Hospitals often possess medical imaging datasets that are too small or institution-specific to support robust computer-vision systems. Sharing raw CT, MRI, X-ray, or pathology images, however, creates privacy and governance problems.
Develop a **Privacy-Preserving Collaborative Medical Imaging Network** in which institutions can collectively train and evaluate computer-vision models without exchanging raw patient images. The system must account for differences between scanners, acquisition protocols, image resolutions, preprocessing pipelines, and patient populations.

Known Constraints

- Keep raw medical images within institutional boundaries.
- Support collaborative model training and evaluation.
- Detect distribution or scanner-specific domain shift.
- Detect anomalous or potentially malicious model updates.
- Track model versions, training provenance, and evaluation evidence.
- Quantify privacy and model-quality trade-offs.

Smart Cities & Urban Infrastructure

Smart Cities & Urban Infrastructure S-01

City-Scale Real-Time Event Fabric

Problem Statement

Modern cities generate large volumes of software-generated events from transportation systems, public services, emergency platforms, municipal applications, public information systems, and other digital infrastructure.
These systems commonly operate as isolated applications. Develop a **City-Scale Real-Time Event Fabric** capable of ingesting, correlating, processing, replaying, and distributing city events across independently developed services.
The platform should act as a software-level real-time coordination layer for a city.

Known Constraints

- Support high-throughput event ingestion.
- Handle duplicate, late, and out-of-order events.
- Provide event replay and historical reconstruction.
- Support schema evolution without breaking consumers.
- Detect and recover from service failures.
- Provide observable event provenance.
- Demonstrate horizontal scalability through load testing.

Smart Cities & Urban Infrastructure S-02

Autonomous Cloud-Edge Computing Fabric

Problem Statement

Large software platforms increasingly execute workloads across centralized clouds, regional compute clusters, and software edge nodes. Workloads are frequently assigned using static deployment policies even though latency, network quality, resource availability, and application demand change continuously.
Develop an **Autonomous Cloud-Edge Computing Fabric** capable of dynamically deciding where software workloads should execute and when workloads should be migrated, replicated, or recovered.

Known Constraints

- Support heterogeneous compute nodes.
- Dynamically place or relocate workloads.
- Handle node failure and network degradation.
- Maintain application state during relocation.
- Expose scheduling decisions and their measurable impact.
- Compare dynamic scheduling against static placement.

Smart Cities & Urban Infrastructure S-03

Urban Infrastructure Cascade Simulator

Problem Statement

Urban services are interconnected. A failure in one digital or operational service can propagate into other services, creating cascading failures that are difficult to detect using isolated monitoring systems.
Develop an **Urban Infrastructure Cascade Simulator** that represents interdependent urban services as a dynamic graph and simulates how disruptions propagate through the system.

Known Constraints

- Represent urban services and dependencies as a dynamic graph.
- Support time-dependent system state.
- Simulate failures and recovery actions.
- Support multiple simultaneous disruptions.
- Measure cascade depth, affected services, and recovery time.
- Provide reproducible simulation scenarios.

Sustainability & Environment

Sustainability & Environment E-01

Decentralized Environmental Data Provenance Network

Problem Statement

Environmental claims increasingly depend on digital datasets such as emission records, carbon accounting data, sustainability reports, environmental assessments, and derived scientific datasets.
Downstream users often cannot determine where a claim originated, whether its evidence was modified, which organization verified it, or which version of the source data produced it.
Develop a **Decentralized Environmental Data Provenance Network** that creates cryptographically verifiable chains from source datasets to published environmental claims.

Known Constraints

- Cryptographically bind claims to source evidence.
- Maintain versioned provenance.
- Support independent verification.
- Handle conflicting or corrected evidence.
- Provide verifiable organizational identities.
- Minimize on-chain storage of sensitive or large datasets.

Sustainability & Environment E-02

Semantic Environmental Data Compression Network

Problem Statement

Large environmental software platforms may generate enormous volumes of temporal data. Transmitting every measurement is inefficient when the underlying system remains within an expected operating range.
Develop a **Semantic Environmental Data Compression Network** in which distributed software nodes communicate meaningful changes in system state rather than blindly transmitting every measurement.

Known Constraints

- Define an application-level reconstruction error bound.
- Detect meaningful deviations from predicted state.
- Support adaptive transmission policies.
- Recover from missing or delayed messages.
- Compare semantic compression against conventional compression.
- Measure bandwidth reduction and reconstruction quality.

Sustainability & Environment E-03

Climate Adaptation Strategy Simulator

Problem Statement

Environmental planning often compares a limited number of predefined strategies. Real-world decision making, however, involves uncertain futures, limited budgets, conflicting objectives, and interactions between infrastructure, population, economy, and environmental conditions.
Develop a **Climate Adaptation Strategy Simulator** capable of exploring large numbers of possible adaptation strategies under uncertain future scenarios.

Known Constraints

- Support multiple future scenarios.
- Model competing objectives and budget constraints.
- Simulate intervention strategies.
- Quantify uncertainty and sensitivity.
- Support large-scale scenario exploration.
- Produce reproducible comparisons between strategies.

Cybersecurity & Digital Trust

Cybersecurity & Digital Trust C-01

Real-Time Software Supply Chain Graph

Problem Statement

Modern software systems depend on thousands of external packages, libraries, container images, CI/CD actions, build systems, and transitive dependencies. A vulnerability or malicious modification in one component can propagate through a large software ecosystem.
Develop a **Real-Time Software Supply Chain Graph** that continuously models software dependencies and determines the potential blast radius of a compromised component.

Known Constraints

- Ingest and normalize software dependency metadata.
- Support SBOM-based representation.
- Track transitive dependencies.
- Correlate vulnerabilities with affected components.
- Calculate blast radius.
- Track dependency changes over time.
- Provide explainable impact paths.

Cybersecurity & Digital Trust C-02

Adaptive Cyber Deception Infrastructure

Problem Statement

Traditional honeypots and decoy services are often static. Once an attacker identifies the deception, its effectiveness decreases.
Develop an **Adaptive Cyber Deception Infrastructure** that continuously modifies its software-defined environment according to observed attacker behavior.

Known Constraints

- Dynamically create and retire decoy services.
- Maintain consistent relationships between decoy resources.
- Detect interaction with deceptive assets.
- Isolate deception from production workloads.
- Preserve forensic evidence.
- Measure attacker engagement and deception effectiveness.

Cybersecurity & Digital Trust C-03

Cryptographic Agility & Post-Quantum Migration Graph

Problem Statement

Organizations frequently do not have a complete inventory of cryptographic algorithms and protocols used across their software infrastructure. A single organization may depend on cryptography across TLS, VPNs, APIs, mobile applications, databases, certificates, identity systems, and blockchain components.
Develop a **Cryptographic Agility and Post-Quantum Migration Graph** that discovers cryptographic dependencies and determines safe migration paths when an algorithm or protocol must be replaced.

Known Constraints

- Discover cryptographic algorithms and protocols.
- Link cryptographic usage to applications and dependencies.
- Identify systems affected by algorithm deprecation.
- Model compatibility constraints.
- Generate migration sequences.
- Detect legacy systems that block migration.
- Provide auditable migration decisions.

Cybersecurity & Digital Trust C-04

Spatial Cyber Threat Reconstruction Engine

Problem Statement

Modern buildings contain hundreds of interconnected devices distributed across multiple floors, network segments, and access zones. Security events are often observed independently, making it difficult to determine how an incident evolved across physical locations and network relationships. A sequence of individually harmless events may represent a coordinated attack when viewed together.
Develop a **Spatial Cyber Threat Reconstruction Engine** for a simulated multi-floor building that continuously combines device telemetry, network relationships, authentication events, and physical location information. The system should maintain a dynamic representation of the environment, correlate events across time and location, reconstruct plausible attack paths, detect potential lateral movement, prioritize threats, and provide an explainable representation of how an incident may have propagated through the building.

Known Constraints

- The environment consists of a multi-floor simulated building with heterogeneous connected devices.
- Device identity, network relationships, and physical location must be represented explicitly.
- Telemetry may be incomplete, duplicated, delayed, or out of order.
- Legitimate anomalous behaviour should be distinguishable from coordinated malicious activity as far as possible.
- The system must maintain temporal relationships between security events.
- Attack paths and threat scores should be explainable rather than black-box outputs.
- Failure or loss of telemetry from part of the environment must not invalidate the entire monitoring system.
- The prototype must operate entirely in a simulated/software environment; no real-world network intrusion or scanning is required.

Enterprise Productivity & Intelligent Automation

Enterprise Productivity & Intelligent Automation P-01

Enterprise Process Genome

Problem Statement

Organizations formally document workflows, but actual work rarely follows those documented processes. Real workflows emerge across source-control systems, email, tickets, chat, documents, approvals, CRM, ERP, incident systems, and meetings.
Develop an **Enterprise Process Genome** that reconstructs how an organization actually operates from distributed digital traces.

Known Constraints

- Ingest heterogeneous enterprise events.
- Reconstruct process execution paths.
- Identify bottlenecks and recurring loops.
- Detect process deviations.
- Model temporal and organizational dependencies.
- Simulate workflow changes.
- Preserve evidence and provenance for discovered processes.

Enterprise Productivity & Intelligent Automation P-02

Distributed Transaction Coordinator for Human Workflows

Problem Statement

Modern business workflows frequently span multiple independent systems such as CRM, payment, inventory, invoicing, approval, and notification services.
Traditional distributed transaction systems assume machine participants that respond quickly and deterministically. Human approvals do not satisfy those assumptions.
Develop a **Distributed Transaction Coordinator for Human-in-the-Loop Workflows**.

Known Constraints

- Support durable workflow execution.
- Implement idempotent operations.
- Support compensation and saga-style recovery.
- Handle human approval checkpoints.
- Manage timeouts and abandoned workflows.
- Recover from partial system failure.
- Maintain complete execution history.

Enterprise Productivity & Intelligent Automation P-03

Natural Language to Verified Workflow Compiler

Problem Statement

Organizations express many business processes and policies in natural language. Humans can interpret instructions such as “verify the vendor, check the budget, obtain finance approval, and create the procurement ticket,” but software cannot safely execute such instructions without formalizing ambiguity and authorization. Develop a **Natural Language to Verified Workflow Compiler** that transforms natural-language business policies into executable workflows.

Known Constraints

- Parse natural-language business policies.
- Convert policies into an intermediate representation.
- Generate executable workflow graphs.
- Detect semantic ambiguity.
- Perform static policy and authorization checks.
- Identify unreachable or circular workflow states.
- Produce human-readable explanations of verification failures.

Think beyond the demo. Research the problem. Engineer the solution.